

Guía de cierre y rúbrica del examen suspenso

Equipo FUVV — SCLI, Sistema de Control de Laboratorios Informáticos

Identificación	
Asignatura	Aplicaciones Distribuidas (ISR-701) — 2026–2027 PPA
Paralelo	7.º “A” Software
Repositorio evaluado	gleiston-guerrero/Entrega-final-del-PFC
Cabecera evaluada	Rama <code>feature/entrega-4</code> , commit <code>d0ef60b</code> — 366 commits — último commit anterior al domingo 13/09/2026, 19:00; verificación sobre clon completo, 16 de septiembre de 2026
Integrantes	Farinango Guandinango Freddy Vladimir · Urbina Romero Isaías Abraham · Villamarín Cuenca Iván Andrés · Vinueza Sánchez Harold Nicolás
Rinden el examen suspenso	Urbina Romero Isaías Abraham · Villamarín Cuenca Iván Andrés
Compleitud del proyecto	64 %
Corte del examen suspenso	Viernes 18 de septiembre de 2026, 23:55 (Ecuador)
Docente	Ing. Gleiston Cicerón Guerrero Ulloa, PhD.

Cláusulas de entrega del examen suspenso (anulan la calificación si no se cumplen)

1. Repositorio transferido y aceptado. El repositorio debe estar transferido a la cuenta del docente (gleiston-guerrero) y la transferencia **aceptada** antes del corte. Si al viernes 18/09/2026 a las 23:55 el repositorio no figura bajo `github.com/gleiston-guerrero/`, el examen se toma como **no entregado** y su nota es cero, cualquiera que sea el trabajo realizado. Estado al 13/09: la transferencia se hizo la noche del 13/09 y al 16/09 el repositorio figura bajo `gleiston-guerrero/Entrega-final-del-PFC`; cumple si sigue así al corte. Además, al corte la rama `main` tenía un único commit del 15/08 y todo el trabajo estaba en `feature/entrega-4`. Todo lo que se califica se revisa en esa URL y en su rama principal; lo que quede en ramas sin fusionar o en cuentas personales no existe para esta evaluación.

2. Carátula en una sola página subida al SGA. Cada estudiante que rinde el examen sube al SGA un único PDF de **una sola página**, sin capturas de cuestionarios, calificaciones ni contenido adicional, con: universidad, facultad y carrera; asignatura, paralelo y periodo; “Examen suspenso — Proyecto Fin de Curso”; nombre completo del estudiante que la sube; equipo y nombre del sistema; integrantes del equipo; docente; URL canónica del repositorio (<https://github.com/gleiston-guerrero/Entrega-final-del-PFC>) escrita completa en una línea y como enlace activo; rama, etiqueta anotada de cierre y hash corto del commit final; fecha de entrega. Una carátula de más de una página, con una URL distinta de la canónica o que no se suba al SGA equivale a examen no entregado para ese estudiante.

1. Evaluación de los entregables

#	Entregable / indicador	Estado propio	Estado efectivo	%	Acción
1	Identificación del entregable y URL	Por modificar	Por modificar	75 %	Véase §1
2	Estructura del repositorio y archivos de raíz	Por modificar	Por modificar	70 %	Véase §2
3	Línea base congelada e integración a <code>main</code>	Por culminar	Por culminar	25 %	Véase §3
4	Backend en capas y SOLID	Por modificar	Por modificar	65 %	Véase §4
5	Patrones GoF y ADR	Por modificar	Por modificar	70 %	Véase §5
6	Aplicación web funcional	Por modificar	Por modificar	75 %	Véase §6
7	Calidad de la aplicación web	Por modificar	Por modificar	80 %	Véase §7
8	Aplicación móvil funcional	Por modificar	Por modificar	70 %	Véase §8

#	Entregable / indicador	Estado propio	Estado efectivo	%	Acción
9	Calidad de la aplicación móvil	Por modificar	Por modificar	60 %	Véase §9
10	Integración y contratos	Por modificar	Por modificar	60 %	Véase §10
11	Persistencia distribuida	Por modificar	Por modificar	75 %	Véase §11
12	Pirámide de pruebas	Por modificar	Por modificar	70 %	Véase §12
13	Pipeline CI/CD	Por modificar	Por modificar	70 %	Véase §13
14	Observabilidad	Por modificar	Por modificar	60 %	Véase §14
15	Protocolo y campaña experimental	Por modificar	Por modificar	60 %	Véase §15
16	Paquete de datos y reproducibilidad	Por modificar	Por modificar	60 %	Véase §16
17	Evaluación ISO/IEC 25010	Por modificar	Por modificar	55 %	Véase §17
18	Manuscrito, bibliografía y trazabilidad E1–E4	Por modificar	Por modificar	55 %	Véase §18
19	Amenazas a la validez y reflexión ética	Por modificar	Por modificar	80 %	Véase §19
20	Trabajo en equipo y autoría	Por modificar	Por modificar	55 %	Véase §20

Cómo leer las dos columnas de estado

El *estado propio* describe el artefacto en sí. El *estado efectivo* aplica la regla de dependencia: **ningún artefacto puede estar más completo que aquello de lo que se construye**. El backend en capas sostiene a los dos clientes; los clientes y el backend sostienen los contratos y la pirámide de pruebas; las pruebas sostienen el pipeline; el sistema instrumentado sostiene la campaña experimental; la campaña sostiene el paquete de datos; el paquete sostiene la evaluación ISO/IEC 25010; y el manuscrito, las amenazas a la validez y la línea base congelan y reportan todo lo anterior. Las actas, la declaración de uso de IA y los archivos de raíz no propagan: se declaran. Lo que se califica es el estado efectivo.

Lo que está hecho y no se recalifica

Ningún entregable está completo, pero hay piezas verificadas que no se tocan. Eficiencia (PI1) recalculada desde los datos crudos de `eficiencia_nominal_50u_5m`, repeticiones 2 a 9, sobre la fila `GET /api/v1/reservas`: p95 media 45,5 ms, IC95 % [27,30; 63,70]; p99 media 371,25 ms, IC95 % [66,91; 675,59], idénticas al documento y con umbrales fijados antes de medir. Campaña ARBITER: 123 de 130 corridas, manifiesto 315/315, censo HTTP de Esc-2 y Esc-3 idéntico al publicado y `analisis-e3.json` regenerado byte a byte. Manifiesto general 468/468 sobre los bytes del repositorio. Validador de contratos en tiempo de ejecución sobre los cinco servicios (12/44/71/79/224 operaciones). Instantánea documental E3 congelada (14 páginas, cero indefinidas). Manuscrito vivo de 33 páginas sin citas ni referencias indefinidas, 29 referencias y 14 DOI resueltos. SPA en TypeScript estricto con `tsc` y lint limpios y 280/280 pruebas; JWT en `sessionStorage` con expiración; serie temporal y calendario semanal sobre la API real. Móvil con `EncryptedSharedPreferences`, Room sin conexión, escáner QR con CameraX y ML Kit, unas 120 pruebas de ViewModel y APK con firma v1/v2/v3 de clave propia. Backend con 869 pruebas, Testcontainers con CockroachDB en tres servicios y JaCoCo ≥ 70 % con compuerta en los cinco; State, Facade, Repository y Factory Method genuinos; registros JSON con `trace_id` en los cinco servicios.

2. Rúbrica del examen suspenso

Cómo se califica el examen suspenso

Sólo se califican los ítems de esta tabla: lo que está en Hecho no vuelve a puntuar ni se puede perder. El peso de cada ítem sale de sumar cuatro estimaciones de lo que cuesta llevarlo a Hecho con la calidad exigida: **profundidad** (P, 1–5: de aplicar una instrucción a rehacer análisis, diseño experimental o arquitectura con criterio propio), **complejidad** técnica (C, 1–5: de trámite mecánico a coordinar varios servicios, datos y herramientas), **amplitud** (A, 1–5: de un archivo a varios módulos, clientes, flujos y documentos a la vez) y **tiempo** de dedicación (T, 1–5: menos de 2, de 2 a 6, de 6 a 15, de 15 a 30 y más de 30 horas-persona), normalizando a 100 dentro del equipo. Cada ítem recibe su peso multiplicado por el coeficiente del estado en que lo encuentre el viernes 18 de septiembre a las 23:55: **Hecho 1,00 · Por modificar 0,40 · Por culminar 0,20 · Por iniciar 0,00**. La nota del examen es la suma de puntos dividida entre diez. Un ítem sólo llega a Hecho si la comprobación descrita en “Cómo lo verifico” da el resultado exigido sobre el repositorio del docente; el mensaje de un commit, un archivo vacío o una cifra sin dato crudo que la respalde no cuentan. Las cláusulas de entrega de la primera página y la ausencia total de la aplicación web o de la móvil (factor 0,5) siguen vigentes sobre la nota.

§	Ítem que se califica	P	C	A	T	Pts	Peso %	Estado 13/09	Puntos
§1	URL canónica, nombre único del sistema y del equipo, autores completos en CITATION.cff	1	1	2	1	5	1,8	Por modificar	0,72
§2	LICENSE, .env.example con web y móvil, retirar el PDF de calificación, mapa del Listado 3 y .gitattributes de datos	1	2	2	1	6	2,2	Por modificar	0,88
§3	Integración a main por PR con revisión sustantiva y CI en verde, etiqueta anotada de cierre y CHANGELOG.md	2	2	3	2	9	3,3	Por culminar	0,66
§4a	Cero dependencias de application hacia infrastructure y presentation (228 imports en 56 archivos)	3	3	4	4	14	5,4	Por modificar	2,16
§4b	Dominio sin tipos Spring/JPA, paquetes heredados de reservas reubicados y reglas ArchUnit en mvn verify	2	2	3	3	10	3,7	Por modificar	1,48
§5a	Observer de alertas con dos o más suscriptores reales y Strategy con dos o más políticas de reserva seleccionables	3	3	3	3	12	4,4	Por culminar	0,88
§5b	Singleton del cliente Prometheus conforme, expiración por el patrón State y ADR-005/006 exactos	2	2	2	2	8	2,9	Por modificar	1,16
§6	Internacionalización de toda la SPA, /settings y /about para todo usuario, inicio real y tema oscuro coherente	1	2	4	3	10	3,7	Por modificar	1,48
§7	Cobertura web $\geq 70\%$ sobre todo src/ y E2E Playwright sin saltos, ejecutadas en verde	2	3	3	3	11	4,0	Por modificar	1,60
§8a	APK release contra el gateway real (sin 10.0.2.2 ni HTTP plano) y QR que abre la reserva rápida	2	2	2	2	8	2,9	Por modificar	1,16

§	Ítem que se califica	P	C	A	T	Pts	Peso %	Estado 13/09	Puntos
§8b	Notificaciones push FCM de extremo a extremo al crear incidentes, con evidencia de recepción	3	4	3	3	13	4,8	Por culminar	0,96
§9a	Pruebas móviles JUnit 5, cobertura ≥ 70 % con compuerta y E2E instrumentada login→lista→detalle	3	3	4	4	14	5,1	Por culminar	1,02
§9b	APK firmado generado desde main tras las pruebas y publicado de forma verificable	2	2	1	1	6	2,2	Por modificar	0,88
§10a	Contratos generados por los clientes reales (web y móvil) y verificados contra el proveedor arrancado, incluido auth	3	3	3	4	13	4,8	Por modificar	1,92
§10b	Validación de esquemas OpenAPI no circular como compuerta de publicación	2	3	2	2	9	3,3	Por modificar	1,32
§11	Experimento de caída de nodos reproducible, verificador de CI corregido y límite de licencia resuelto o justificado	3	3	2	3	11	4,0	Por modificar	1,60
§12	Informes de las cinco capas de pruebas como artefactos y carga nominal y rampa repetidas con el harness corregido	2	3	3	3	11	4,0	Por modificar	1,60
§13	Pipeline completo en verde en el commit final, lint en los cinco servicios y compuertas completas antes de publicar	2	3	3	2	10	3,7	Por modificar	1,48
§14a	Seis paneles con datos bajo carga, cuatro métricas nuevas en los servicios y traza distribuida exportada	2	3	3	3	11	4,0	Por modificar	1,60
§14b	Latencia extremo a extremo medida en el cliente móvil y propagada con traceparent	3	4	3	3	13	4,8	Por iniciar	0,00
§15a	Campaña correctiva de fiabilidad 10×1 h ejecutada y versionada, sin 401 inducidos por el instrumento	3	3	2	4	12	4,4	Por culminar	0,88
§15b	ARBITER: n suficiente en Esc-1, escenarios que ejerciten I4 e I5 y censo HTTP de todos los escenarios	4	3	3	3	13	4,8	Por modificar	1,92
§16	Manifiestos que verifican en cualquier clon y en CI, CSV canónico único y datos de toda ejecución citada	2	2	3	3	10	3,7	Por modificar	1,48
§17	ISO/IEC 25010 sobre la población válida: fiabilidad corregida, complejidad ciclomática, seguridad por endpoint y compatibilidad API 26/34	3	3	3	3	12	4,4	Por modificar	1,76
§18	Manuscrito: estructura de la Tabla 3, resúmenes ≥ 200 palabras, capturas exigidas, PDF versionado, instantánea E4 congelada y cifras canónicas	2	2	3	3	10	3,7	Por modificar	1,48
§19	Amenazas de sesión y licencia con mitigación real y reflexión ética $\geq \frac{1}{2}$ página	2	1	1	1	5	1,8	Por modificar	0,72
§20	Actas contemporáneas, PR revisados por otro integrante y declaración de uso de IA con alcance	1	1	2	2	6	2,2	Por modificar	0,88
Total							100		

Lectura de la rúbrica al 13/09/2026, 19:00

La columna *Estado* es el que encontré en el commit de corte; la columna *Puntos* es peso por coeficiente. Nota provisional del examen suspenso: **3,37 / 10,00**. Si cierran todos los ítems en Hecho, la nota es 10,00.

3. Trabajo exigido

§1 Identificación del entregable y URL

Qué encontré: la URL del propietario anterior (ffarinangog2/Entrega-final-del-PFC) sigue en ocho archivos, entre ellos CITATION.cff, README.md, docs/main.tex (\RepoURL), docs/entrega-4/main.tex y docs/actas/README.md. El sistema tiene tres nombres (“Laboratorios e Infraestructura”, “Laboratorios e Informática”, “Laboratorios Informáticos”), el equipo aparece como “ForáCode” en la portada y como FUVV en el resto, y CITATION.cff lista un nombre y un apellido por autor, sin roles. **Cómo debe quedar:** <https://github.com/gleiston-guerrero/Entrega-final-del-PFC> en README, CITATION.cff y documentos vivos; un solo nombre de sistema y de equipo; nombres completos y roles de los cuatro integrantes. **Cómo lo verifico:** `git grep -n "ffarinangog2/Entrega" HEAD -- README.md CITATION.cff docs/main.tex` no devuelve nada.

§2 Estructura — licencia, variables, raíz y finales de línea

Qué encontré: no existe LICENSE; .env.example no tiene variables de la web ni del móvil; el commit de corte versiona un PDF con la calificación personal de un integrante en el cuestionario del SGA, ajeno al proyecto; faltan docs/api/openapi.yaml, protocolo-e3.md y boxplot_latencia.png del Listado 3 sin que el README explique la equivalencia; y .gitattributes sólo fija eol=lf para dos rutas, de modo que en un clon de Windows fallan 373 de 468 sumas del manifiesto general, 50 de 50 de evidencia-e3-canónica y la del APK. **Qué deben hacer:** añadir LICENSE; declarar VITE_API_BASE_URL y SCLI_API_BASE_URL en .env.example; retirar el PDF del cuestionario; añadir al README una tabla “Listado 3 → ruta real”; declarar experimentos/** -text y release/apk/SHA256SUMS.txt -text y renormalizar. **Cómo lo verifico:** `test -f LICENSE; git ls-files | grep -i cuestionario` vacío; en un clon con `core.autocrlf=true` y otro con `false`, `sha256sum -c` no imprime ningún fallo.

§3 Línea base — nada integrado ni congelado al corte

Qué encontré: al 13/09 a las 19:00 main tenía un único commit (859bc58, “Entrega anterior”, 15/08). El PR que integra feature/entrega-4 tiene 367 commits, 2.417 archivos y una sola aprobación de una frase genérica, sin un comentario en línea, emitida con el pipeline de ese commit en rojo; la fusión la hizo el propio autor a las 21:28, fuera de plazo y con dos commits que la aprobación no cubre. No hay ninguna etiqueta ni CHANGELOG.md. **Cómo debe quedar:** todo el trabajo en main mediante PR cuyo revisor, distinto del autor, deja comentarios sobre el código y aprueba con el pipeline en verde sobre el commit final; CHANGELOG.md con una entrada fechada por entrega (E1 a E4) y una del examen suspenso; etiqueta anotada de cierre sobre el último commit de main, creada al final de todo. **Cómo lo verifico:**

```
git cat-file -t v4.0.0          # debe imprimir: tag
git merge-base --is-ancestor v4.0.0 origin/main && git rev-parse v4.0.0~{commit} origin/main
git ls-remote --tags origin v4.0.0    # deben salir DOS líneas
```

y en la API de GitHub, por cada PR del suspenso, /reviews con un aprobador distinto del autor y /comments con al menos un comentario sobre el código.

§4 Backend en capas — la capa de aplicación depende de las externas

Qué encontré: 71 imports de `application` hacia `infrastructure` y 157 hacia `presentation`, en 56 archivos de cuatro servicios; por ejemplo `AdminUsuarioService` usa directamente `UsuarioAuthRepository` saltándose su propio puerto, y `NotificacionService` usa un repositorio JPA. Los diez puertos de académico importan `Page` y `Pageable` de Spring; `BloqueoAgendaRepositoryPort` expone una entidad con `@Entity`; reservas conserva los paquetes `entity`, `repository`, `client`, `mapper` y `security` fuera de las capas; y nada lo hace cumplir en Cl. **Cómo debe quedar:** cero dependencias de `application` hacia `infrastructure` o `presentation`; dominio sin tipos de Spring ni de JPA; reglas `layeredArchitecture()` de ArchUnit por servicio ejecutadas en `mvn verify`. **Qué deben hacer:** mover los DTO a `application/dto` y mapear en el controlador; sustituir los repositorios JPA usados en aplicación por puertos; crear un tipo de paginación propio del dominio en académico (usuarios ya lo tiene) y un modelo de dominio para `BloqueoAgenda`. **Cómo lo verifico:**

```
grep -rEn "import .*\.(infrastructure|presentation)\." services/*/src/main/java --include=*.java | grep
↪ /application/ | wc -l # 0
grep -rEn "org\.springframework\.jakarta\.persistence" services/*/src/main/java --include=*.java | grep
↪ /domain/ | wc -l # 0
grep -rL "layeredArchitecture" services/*/src/test | wc -l
↪ # >= 4
```

§5 Patrones GoF — tres de los cinco exigidos no cumplen

Qué encontré: State, Facade, Repository y Factory Method son genuinos. El “Singleton del cliente Prometheus” (`HttpRequestsMetricsRegistry`) es un `@Component` con constructor público que asigna `instance = this`, envuelve `MeterRegistry` y está duplicado en usuarios; el cliente real, `PrometheusQueryClient`, es un bean normal. El Observer “de alertas” no existe: sólo hay un oyente de perfil que escribe en el log, y la palabra “alert” no aparece en el código de producción. La Strategy “de políticas de reserva” tiene una única implementación fija, y la estrategia real (`ArbitrajeStrategy S0–S4`) está desactivada en operación y no figura en el ADR. `SolicitudExpirationScheduler` cambia el estado con un `setEstado` que se salta el patrón State. ADR-006 usa el estado “Implementado”, ajeno al formato Nygard, y sus dos criterios cuantitativos no comparan ninguna cifra con Flutter o React Native. **Cómo debe quedar:** Observer de alertas o incidentes con dos o más suscriptores reales (notificación push y auditoría o métrica); dos o más políticas de reserva seleccionables en ejecución y probadas; singleton conforme (constructor privado o justificación explícita del singleton del contenedor) sin copias; expiración por el patrón State; ADR-005 con los conteos reales y ADR-006 en formato Nygard con una comparación cuantitativa entre alternativas y fuentes fechadas. **Cómo lo verifico:** `grep -rn "implements .*Strategy" services/reservas-solicitudes-service/src/main | wc -l ≥ 2` con su prueba de selección; dos o más clases suscriptoras del sujeto de alertas; lectura de ambos ADR.

§6 Aplicación web — internacionalización aparente

Qué encontré: las cinco rutas, la ruta protegida, el JWT con expiración, la serie temporal y el calendario funcionan, y `es.json` y `en.json` tienen las mismas 57 claves; pero sólo 3 de 38 componentes usan `t()` y quedan unos 283 textos fijos en español (inicio de sesión, calendario, monitoreo, menú, mensajes de error), de modo que al cambiar a inglés casi toda la aplicación sigue en español. `/settings` y `/about` exigen el rol de administrador; `/` sólo redirige a `/login`; el tema oscuro es un único bloque CSS frente a 233 colores fijos; la serie temporal sólo la ve el administrador. **Cómo debe quedar:** toda la interfaz traducida con fechas en el idioma activo; `/settings` y `/about` para cualquier usuario autenticado; `/` como página de inicio; tema oscuro coherente en todas las pantallas mediante variables CSS. **Cómo lo verifico:** la prueba E2E de idioma y tema, ampliada, cambia a inglés y comprueba textos en `/login`, `/main` y el calendario; `grep -rL useTranslation apps/web/src --include=*.tsx | grep -v test` sólo lista componentes sin texto visible.

§7 Calidad web — cobertura sobre un alcance favorable y E2E que no corrieron

Qué encontré: lint y tsc limpios, 280/280 pruebas y Dockerfile multi-stage. La cobertura declarada (ramas 73,69 %) se mide sólo sobre los archivos que importan las pruebas porque vite.config.ts no define coverage.include; con include: src/** las ramas caen a 68,94 % y el umbral falla, con LoginPage, ProfilePage, PlanificacionPage, router/index.tsx y App.tsx al 0 %. Esa cifra mal acotada es la que llega a la tabla de mantenibilidad. En Playwright, una prueba se salta con test.skip porque ningún flujo define sus variables, otra está excluida en la configuración, las seis restantes no tocan la serie temporal ni el calendario, y en el commit de corte ninguna se ejecutó porque falló el paso previo del job de integración. Hay credenciales escritas en auth.spec.ts. **Cómo lo verifico:**

```
cd apps/web && npm ci && npx vitest run --coverage --coverage.include='src/**/*.ts,tsx' # código de salida
↪ 0
grep -rn "test.skip" apps/web/e2e # vacío
```

y el job de integración del commit final en verde con el recuento de pruebas Playwright en su registro.

§8 Aplicación móvil — el APK de entrega no puede hablar con el sistema

Qué encontré: Kotlin con Compose y MVVM, API mínima 26 y objetivo 34, inicio de sesión por el gateway, listado y detalle con actualización por arrastre y caché Room, y QR real. Pero el .dex del APK versionado contiene http://10.0.2.2:8080/ (el CI no define SCLI_API_BASE_URL) y el tráfico HTTP sin cifrar sólo se permite en el manifiesto de depuración, así que en release queda bloqueado. Las notificaciones push no funcionan: no hay google-services.json, el APK no contiene google_app_id, el backend tiene FIREBASE_ENABLED=false y el propio README lo admite. Crear un incidente no notifica a ningún técnico (sólo hay una notificación local), y el detalle del laboratorio escaneado no lleva a la reserva rápida que pide la Tabla 1. **Cómo debe quedar:** APK de release contra el gateway por HTTPS o con network_security_config restringido; push FCM de extremo a extremo desde la creación del incidente hasta los técnicos con permiso, con evidencia versionada (identificador del mensaje enviado, registro del dispositivo y captura); acción “Reservar este laboratorio” tras escanear. **Cómo lo verifico:** unzip -p release/apk/*.apk 'classes*.dex' | strings | grep -c 10.0.2.2 da 0; los recursos del APK contienen google_app_id; la creación de incidentes llama al puerto de notificación.

§9 Calidad móvil — sin prueba extremo a extremo y con cobertura del 38 %

Qué encontré: 291 pruebas unitarias, unas 120 de ViewModels, todas con JUnit 4; las siete instrumentadas prueban pantallas con dobles, un DAO y una migración, y ninguna recorre la aplicación (actividad, inicio de sesión, navegación, red). Cobertura de líneas recalculada desde jacocoTestReport.xml: 1.733/4.520 = 38,34 %, sin compuerta. El APK versionado se subió a mano el 12/09; su firma es válida con clave propia, pero nada lo vincula con el artefacto del CI; el job firmado sólo corre al empujar a feature/entrega-4 (en el PR salió skipped) y no depende de las pruebas. **Cómo debe quedar:** ViewModels con JUnit 5; cobertura ≥70 % con jacocoTestCoverageVerification; al menos una E2E instrumentada con createAndroidComposeRule<MainActivity> que recorra inicio de sesión, lista, detalle y modo sin conexión contra un servidor simulado; APK firmado generado desde main después de las pruebas y publicado de forma verificable (Release con su suma citada en el README o atestación de procedencia). **Cómo lo verifico:** grep -rl org.junit.jupiter apps/mobile/app/src/test | wc -l; grep -rn "createAndroidComposeRule<MainActivity>" apps/mobile/app/src/androidTest; la suma publicada coincide con sha256sum del APK descargado del CI.

§10 Contratos — escritos a mano y verificados sin el proveedor real

Qué encontré: los contratos consumidores están escritos a mano en Java dentro de `tests/contract`; no se generan desde la web (no hay `pact-js`) ni desde el móvil, y ya están desfasados: la app pide `GET api/v1/incidentes?tamano=100` y el contrato define ese GET sin parámetros. La verificación del proveedor usa `standaloneSetup` con servicios simulados, sin contexto de Spring ni seguridad, y `auth-service` no tiene prueba de proveedor, así que sus contratos nunca se verifican. El validador en tiempo de ejecución es una mejora real, pero sólo compara método y ruta, y el job `validate-openapi-contracts` sigue ejecutando el script que compara el JSON con su propio extractor por expresiones regulares, lo cual es circular. Además esos cambios rompieron el pipeline (§13). **Cómo debe quedar:** contratos generados por los clientes reales (`pact-js` sobre la capa de API web; pruebas consumidoras sobre las interfaces Retrofit), verificados contra el proveedor arrancado (`@SpringBootTest(webEnvironment = RANDOM_PORT)`) en cada servicio consumido, incluido `auth`, y validación de esquemas contra `/v3/api-docs` como compuerta de la publicación. **Cómo lo verifico:** `grep -n pact apps/web/package.json apps/mobile/app/build.gradle.kts; grep -rn standaloneSetup` en las pruebas de proveedor devuelve vacío; existe la prueba de proveedor de `auth`; job de verificación en verde en el commit final.

§11 Persistencia distribuida — tolerancia a fallos sin experimento reproducible

Qué encontré: tres nodos CockroachDB con TLS, `num_replicas=3`, Flyway en cuatro servicios y un script de CI que detiene un nodo (en verde). Pero la evidencia de tolerancia a fallos son capturas del 28/07 de otra topología (`--insecure`) más un vídeo de 24,6 MB, con latencias de una sola medición dominadas por el arranque del cliente, y cita una figura que no existe. Con el nodo caído el script sólo ejecuta `SELECT 1` y compara la versión de migración como texto, de modo que nunca verifica V10–V14. `Auth`, usuarios y académico corren en nodo único. Y el registro del servicio de reservas de la campaña de fiabilidad muestra “No license installed. The maximum number of concurrently open transactions has been reached”, que es el origen de los HTTP 500 de las campañas. **Cómo debe quedar:** carpeta `experimentos/crdb-failover/` con script, estado del clúster antes, durante y después, lecturas y escrituras con marca UTC, cinco o más repeticiones con uno y dos nodos caídos y manifiesto; verificador de CI que lea y escriba en tablas replicadas con el nodo caído y compare la versión de migración como entero; límite de licencia resuelto antes de volver a medir, o justificado en un ADR junto con la decisión de nodos únicos.

§12 Pirámide de pruebas — capas escritas pero no ejercitadas al corte

Qué encontré: 869 pruebas de backend, Testcontainers con CockroachDB en tres servicios y el `locustfile` ya renueva el testigo. Pero en el commit de corte falló el job de integración, de modo que Playwright y el *smoke* de Locust no se ejecutaron; no hay E2E móvil (§9); las diez corridas nominales y la única rampa son del 30/08, anteriores a la corrección del testigo, y la rampa registra `status: failed`; el CI no publica los informes de Playwright ni de JaCoCo; académico prueba integración con H2 y Flyway desactivado; Testcontainers usa CockroachDB v24.3.5 y el compose v26.2.0. **Cómo debe quedar:** las cinco capas en verde en el commit final, cada una con su informe como artefacto descargable; carga nominal (50 usuarios, 5 minutos) y rampa (0 a 200 en 10 minutos, tres o más corridas) repetidas con el *harness* corregido y sus datos crudos versionados; integración de académico sobre CockroachDB con migraciones reales y la misma versión que el compose.

§13 Pipeline CI/CD — en rojo en el commit de corte

Qué encontré: los siete jobs existen en un DAG coherente, sin `continue-on-error` ni pruebas saltadas, con sonda de salud y seis imágenes etiquetadas por commit. Pero hubo 11 commits consecutivos en rojo hasta el de corte, que no tiene imágenes publicadas. La causa, reproducida

en local: build-helper-maven-plugin añade fuentes de ../../tests/openapi-runtime, fuera del contexto de Docker, y jackson-databind tiene alcance test. El lint del backend sólo corre en académico con seis reglas; ESLint admite advertencias; validate-openapi-contracts y el job del APK firmado no bloquean la publicación de imágenes; el despliegue nunca se ejecutó. **Cómo debe quedar:** ejecución completa en verde sobre el commit etiquetado, con sus imágenes en GHCR; Checkstyle con failOnViolation en los cinco servicios; eslint . --max-warnings 0; todas las validaciones en los needs de build-images. **Cómo lo verifico:**

```
curl -s "https://api.github.com/repos/gleiston-guerrero/Entrega-final-del-PFC/actions/runs?head_sha=$(git
↪ rev-parse v4.0.0~{commit})"
grep -c "checkstyle:check" .github/workflows/ci-cd.yml # >= 5
```

§14 Observabilidad — la mitad del tablero vacío

Qué encontré: registros JSON con trace_id en los cinco servicios, app_business_events_total en flujos reales, app_active_sessions en auth, Collector hacia Tempo y Loki, y un tablero versionado con seis paneles. Pero la captura del propio equipo, tomada bajo carga, muestra “No data” en tres: disponibilidad del clúster (el scrape no declara scheme: https ni tls_config y el clúster usa certificados), recursos por contenedor (cAdvisor sin privilegios) y latencia desde el móvil, que en realidad es un indicador del gateway y tampoco tiene datos. La app no mide ni exporta latencia; su TraceId.kt es un UUID por proceso sin traceparent. http_requests_total sólo existe en dos servicios, http_request_duration_seconds es una regla de grabación sólo para reservas, el tablero no usa ninguna de las cuatro métricas nuevas y no hay una traza distribuida exportada. **Cómo debe quedar:** los seis paneles con series bajo carga Locust, con captura y respuestas query_range versionadas en ops/evidencia/; interceptor en el cliente móvil que mida la duración de cada petición, genere traceparent y exporte la métrica por OTLP; las cuatro métricas en todos los servicios que atienden tráfico y usadas en el tablero; una traza del móvil a CockroachDB exportada y correlacionada con su registro en Loki.

§15 Campaña experimental — la corrección de fiabilidad no se ejecutó

Qué encontré: PI1 quedó bien corregida (véase el recuadro de lo hecho). Fiabilidad no: el protocolo correctivo registra diez repeticiones de una hora con renovación de sesión, pero resultados/iso25010-correctiva.csv tiene sus diez filas vacías; la única repetición correctiva que el cierre declara “válida” y los tres intentos de la segunda no están en el repositorio (el documento lo admite), y lo único versionado es un smoke de 25 minutos. La campaña que sostiene PI2 sigue con 75,05 % de respuestas 401 inducidas por el instrumento, marcada valida=si. En ARBITER, Esc-1 perdió 7 de 10 corridas ($n = 3$ frente a $r = 10$); los invariantes I4 e I5 quedan NOT_OBSERVED en 96 de 96 casos, de modo que el oráculo sólo evalúa tres de cinco; y el censo HTTP omite Esc-4, donde S4 tuvo 351 de 1.600 errores (21,94 %) frente a 0 de S3, porque auditar_http_arbiter.py fija COMPARISON_SCENARIOS=("esc2", "esc3"). Seguridad, mantenibilidad y compatibilidad usan $r = 3$. **Cómo debe quedar:** campaña correctiva de fiabilidad 10×1 h ejecutada después de resolver el límite de transacciones (§11), con datos crudos, metadatos y sumas de cada repetición versionados; ARBITER con repeticiones suficientes en Esc-1 (o exclusión declarada en el protocolo antes de ejecutar), corridas de cancelación y acceso que ejerciten I4 e I5, y censo HTTP de todos los escenarios. **Cómo lo verifico:**

```
python -c "import csv;print(sum(1 for r in
↪ csv.DictReader(open('experimentos/resultados/iso25010-correctiva.csv')) if r['total_requests']))" # 10
grep -rho NOT_OBSERVED experimentos/resultados/arbiter/campaign/analysis/oracle | wc -l
grep -n COMPARISON_SCENARIOS experimentos/*.py
```

§16 Paquete de datos — sumas que no verifican y dos juegos de percentiles

Qué encontré: el manifiesto general verifica 468/468 sobre los bytes del repositorio y ARBITER 315/315, pero evidencia-e3-canonica/MANIFEST-SHA256.txt no verifica en ningún modo de checkout

(falla 43 de 50 sobre los bytes: 43 sumas se calcularon sobre la versión CRLF), el *smoke* de fiabilidad falla 3 de 7 y ningún job del CI verifica los manifiestos. No hay un juego canónico: *iso25010.csv* guarda los percentiles de la fila agregada (r1: p95 110, p99 2.200 ms) y el manuscrito publica los de la fila GET (r1: 87 y 1.000 ms), que no están en ningún CSV procesado. **Cómo debe quedar:** manifiestos regenerados sobre los bytes versionados y protegidos por *.gitattributes*; un paso del CI que ejecute `sha256sum -c` para cada manifiesto; un único CSV canónico con exactamente los valores que publica el documento; datos crudos de toda ejecución que el documento cite. **Cómo lo verifico** (en un clon limpio, en Linux y en Windows):

```
sha256sum -c experimentos/resultados/SHA256SUMS | grep -vc ': OK$'
↪ # 0
(cd experimentos/resultados/evidencia-e3-canonica && sha256sum -c MANIFEST-SHA256.txt | grep -vc ': OK$')
↪ # 0
(cd experimentos/evidencia-e2/smoke-refresh-25m && sha256sum -c SHA256SUMS.txt | grep -vc ': OK$')
↪ # 0
```

§17 Evaluación ISO/IEC 25010 — una respuesta afirmativa sobre una población inválida

Qué encontré: la tasa de 5xx publicada (0,061 %, IC95 % [0,032; 0,091]) se calcula sobre una población donde el 75 % son 401 que no llegan al servicio; con el denominador sin 401, recalculado desde *locust_failures.csv*, es 0,246 % [0,128; 0,364], cuatro veces más, y en el *smoke* con sesión renovada llega a 0,60 % (219 HTTP 500 en 36.331 GET), cerca del umbral del 1 %. Los p95/p99 de fiabilidad usan la fila agregada, con una regla de población distinta de la de P11. Mantenibilidad no mide la complejidad ciclomática que exige la Tabla 2 y usa la cobertura web mal acotada (§7). Seguridad cuenta 7 decisiones × 3 en vez del porcentaje de endpoints con verificación JWT sobre las 224 operaciones del gateway. Compatibilidad móvil “API 26+” sólo se probó en un emulador API 29. **Cómo debe quedar:** las cinco características con la métrica de la Tabla 2, sobre la población que declara el protocolo: fiabilidad desde la campaña correctiva de §15; mantenibilidad con cobertura de todo el código y complejidad ciclomática media por componente (PMD, Checkstyle o *lizard*); seguridad como porcentaje de endpoints protegidos del inventario OpenAPI con una prueba 401 por endpoint; compatibilidad móvil en API 26 y 34. Las métricas deterministas se reportan como medición única, sin intervalo.

§18 Manuscrito — dos introducciones y conclusiones que no son las finales

Qué encontré: compila a 33 páginas sin indefinidas y la bibliografía cumple. Pero no hay PDF versionado de la fuente oficial; *docs/entrega-4*, presentado como instantánea, incluye las secciones vivas, compila con una figura ausente (*panel-monitoreo.png*) y una referencia indefinida (*sec:contratos-openapi*), y su PDF es del 05/09. Hay dos secciones “Introducción”; las secciones de E4 se insertan antes de la discusión y las conclusiones de E3, de modo que el documento termina con las conclusiones de E3; la tabla de módulos de E3 sigue diciendo “Métricas Prometheus: No iniciado” y “Protocolo: sin repeticiones ejecutadas”; el resumen dice que las evidencias “aún deban ejecutarse”, no da ningún resultado y tiene 154 palabras en español y 97 en inglés. No hay capturas de las cinco rutas web, de la app móvil ni de una traza (las “figuras” de las aplicaciones son bloques de texto ASCII). **Cómo debe quedar:** un documento acumulativo con la estructura de la Tabla 3 de la guía (una introducción; discusión y conclusiones finales que integren E1–E4), resúmenes de al menos 200 palabras con los resultados, capturas reales de las cinco rutas en español e inglés, de la app y de una traza en Tempo; cifras idénticas al CSV canónico (§16–§17) y compilado *después* de la última regeneración de resultados; *docs/main.pdf* versionado e igual a la compilación de su fuente; *docs/entrega-4* congelado con copia propia de secciones y figuras, como ya se hizo con *docs/entrega-3*. **Cómo lo verifico:**

```
cd docs && latexmk -pdf -interaction=nonstopmode main.tex && grep -c "undefined\|not found" main.log #
↪ 0
cd entrega-4 && latexmk -pdf -interaction=nonstopmode main.tex && grep -c "undefined\|not found" main.log #
↪ 0
```

```
grep -rc '\\section{Introducci' ../main.tex secciones/*.tex
↪ total 1
```

#

Además extraigo el texto del PDF y busco cada valor de la tabla ISO en el CSV canónico.

§19 Amenazas a la validez y reflexión ética

Qué encontré: tres amenazas internas, dos externas y tres de constructo con el formato amenaza, efecto, mitigación y riesgo. Dos “mitigaciones” internas sólo informan (“se conservan y publican todos los intentos”, “se reporta separada”), y faltan las dos amenazas que más afectan a los resultados: el testigo no renovado (75 % de 401) y el límite de transacciones de CockroachDB sin licencia, causa de los HTTP 500. La reflexión ética cita el Código de Ética de ACM pero tiene 342 palabras, cerca de un tercio de página. **Cómo debe quedar:** ambas amenazas añadidas con una mitigación que las reduzca (campaña correctiva, licencia o ajuste del límite); las dos mitigaciones reescritas como control, repetición o medición adicional; reflexión de al menos media página vinculada a principios numerados del código (1.2, 1.6, 2.5, 3.1) aplicados a los datos de reservas y monitoreo.

§20 Trabajo en equipo y autoría

Qué encontré: las actas se reetiquetaron honestamente como “registros retrospectivos” escritos el 12/09 que “no demuestran reuniones”: no existe ningún acta contemporánea. La revisión cruzada se reduce a la aprobación descrita en §3. La declaración de uso de IA es genérica y no dice en qué artefactos se usó. Urbina Romero tiene 73 commits con aporte real en web, usuarios, móvil, Observer y ADR-006; Villamarín Cuenca tiene 91, pero la mitad de sus líneas son JSON de OpenAPI generados y, entre el 12/09 y el corte, un solo cambio sustantivo. **Cómo debe quedar:** un acta por reunión del periodo del suspenso, versionada el mismo día; cada ítem de esta rúbrica entregado en un PR cuyo autor es uno de los dos integrantes que rinden el examen y cuyo revisor es otro integrante, con comentarios sobre el código; declaración de IA con herramienta, tareas y cómo se verificó lo producido. El aporte de cada estudiante que rinde el examen se acredita sólo por sus propios commits y PR sobre los ítems de esta guía. **Cómo lo verifico:** `git log --format='%h%cI%aN' -- docs/actas/` frente a la fecha interna de cada acta; `git shortlog -sn --no-merges d0ef60b..v4.0.0; /pulls?state=all, /reviews y /comments` de la API.

4. Orden de ejecución

§13 pipeline en verde → §2 y §16 finales de línea, manifiestos y compuerta → §11 límite de transacciones → §15 campañas → §17 ISO/IEC 25010 y CSV canónico → §4 capas y ArchUnit → §5 patrones → en paralelo §6–§7 web y §10 contratos (Villamarín Cuenca) y §8–§9 y §14 móvil y observabilidad (Urbina Romero) → §12 pirámide → §18 manuscrito y §19 amenazas con las cifras definitivas → §1 URL → §20 y §3 PR revisados, fusión a `main`, CHANGELOG, manifiestos finales y, lo último, la etiqueta anotada.

5. Lista de verificación antes de entregar

```
git ls-remote https://github.com/gleiston-guerrero/Entrega-final-del-PFC HEAD # repositorio transferido y
↪ accesible
git cat-file -t v4.0.0 && git merge-base --is-ancestor v4.0.0 origin/main && test -f CHANGELOG.md -a -f LICENSE
sha256sum -c experimentos/resultados/SHA256SUMS | grep -vc ': OK$'
python experimentos/analizar_iso25010.py experimentos/resultados/iso25010-correctiva.csv
grep -rEn "import .*\.(infrastructure|presentation)\." services/*/src/main/java --include=*.java | grep -c
↪ /application/
cd apps/web && npx vitest run --coverage --coverage.include='src/**/*.ts,tsx'
cd docs && latexmk -pdf main.tex && grep -c "undefined\|not found" main.log
```

Documento de uso académico. Aplicaciones Distribuidas (ISR-701), UTEQ, 2026–2027 PPA. Verificación ejecutada sobre clon completo del repositorio declarado, con el estado al domingo 13 de septiembre de 2026 a las 19:00, el 16 de septiembre de 2026. Ing. Gleiston Cicerón Guerrero Ulloa, PhD.